



**Department of Information Technology
Fr. Conceicao Rodrigues Institute of Technology
(An Autonomous Institute & Permanently Affiliated to University of Mumbai)
Sector 9A, Vashi, Navi Mumbai – 400703**

**University of Mumbai
2025-2026**

**SOFTWARE DEVELOPMENT LABORATORY
Prof. Lakshmi Gadikhar**

**CASE STUDY: SARKARI SAHAYATA
By
Harshit Singh - 5024162**

EXPERIMENT 01

Aim: Select a problem statement and write case study on Software Development Life Cycle for the same.

CO: Learner will be able to use concepts of software engineering process models to optimize the flow of project development process.

Problem Statement:

Citizens often face difficulties in identifying and understanding government schemes that they are eligible for due to scattered information, complex eligibility criteria, and lack of digital awareness. SarkariSahayata aims to provide a smart web-based platform that allows users to enter personal details and automatically check their eligibility for government schemes.

Objective:

To implement a structured SDLC approach to design and develop SarkariSahayata, ensuring security, scalability, and user-friendliness.

1. Requirements Gathering and Analysis

Functional Requirements:

1. Allow users to enter personal details.
2. Allow document uploads (Aadhar, PAN, Photo).
3. Check eligibility against scheme database.
4. Display eligible schemes.

Non-Functional Requirements:

1. User-friendly interface.
2. Secure data handling.
3. Fast system response.
4. Scalable architecture.

2. System Design

Frontend: HTML, CSS, Bootstrap

Backend: Python (Flask)

Database: SQLite and CSV

Components include UI, eligibility engine, file upload module, and scheme database.

3. Development

Frontend pages designed using HTML and CSS.

Backend logic implemented using Flask.

Scheme data stored in SQLite database.

4. Testing

Unit, Integration, UI, Security, and Performance testing conducted.

5. Deployment

Flask server configured and deployed.

User documentation provided.

6. Maintenance and Updates

Regular bug fixing and scheme updates.

Future scope includes multilingual support and chatbot integration.

Conclusion:

SarkariSahayata demonstrates effective use of SDLC to create a transparent and efficient platform that improves access to government welfare schemes and promotes digital inclusion.

SDG Mapping:

SDG 11 – Sustainable Cities and Communities

SDG 3 – Good Health and Well-being

SDG 16 – Peace, Justice and Strong Institutions

EXPERIMENT 02

Aim: Implement Waterfall Model and Agile model for the selected problem statement.

CO: Learner will be able to Use concepts of software engineering process models to optimize the flow of project development process.

Theory:

Waterfall Model

The **Waterfall Model** is a linear and sequential SDLC approach. In this model, each phase must be completed before the next one begins, with no possibility of returning to a previous phase. It is best suited for projects with clearly defined and stable requirements.

Advantages

1. **Simple and Easy to Manage:**
Each phase has clearly defined deliverables and review processes.
2. **Clear Milestones:**
Progress is easy to track through well-defined stages.
3. **Well-Documented:**
Extensive documentation ensures transparency and clarity.

Disadvantages

1. **Inflexible:**
Changes are difficult once the project progresses beyond initial stages.
2. **Late Testing:**
Errors may be discovered late as testing occurs after development.
3. **High Risk:**
Not suitable for complex or evolving projects.

Applications

1. Short projects with fixed requirements
2. Regulated industries (government, healthcare)
3. Projects using stable technologies

Implementation of Waterfall Model in Sarkari Sahayata

1. Requirements Gathering and Analysis

- Functional: User data entry, document uploads (Aadhar, PAN), eligibility checks
- Non-Functional: Security, scalability, user-friendly interface
- Deliverable: Software Requirement Specification (SRS)

2. System Design

- Tech Stack: HTML, CSS, Bootstrap (Frontend), Python Flask (Backend)
- Database: SQLite with CSV schema
- Modules: UI, Eligibility Engine, File Upload System

3. Implementation (Development)

- Frontend developed using HTML and CSS
- Backend logic implemented using Flask
- Database populated with scheme data

4. Integration and Testing

- Types: Unit, Integration, UI, Security, Performance Testing
- Objective: Validate eligibility logic and system functionality

5. Deployment

- Flask server deployment
- User and technical documentation provided

6. Maintenance

- Regular updates and bug fixes

- Future enhancements (multilingual support, chatbot integration)

Agile Models

Agile is an iterative development approach that emphasizes continuous feedback, collaboration, and incremental delivery.

Types of Agile Models

1. Extreme Programming (XP) Model

Focuses on technical excellence and frequent releases to improve software quality and adaptability.

2. Scrum Model

A framework where development is divided into fixed-length iterations called **Sprints** (2–4 weeks), delivering functional increments of the system.

3. Kanban Model

A visual workflow management system that focuses on continuous delivery without fixed iterations.

Implementation of Sarkari Sahayata using Scrum Model

The Scrum Team

- **Product Owner:** Manages scheme priorities and ensures user needs are met
- **Scrum Master:** Facilitates the process and resolves team issues
- **Developers:** Build frontend (HTML/CSS/Bootstrap) and backend (Flask)

Sprint 1: Foundation & User Data

Goal: Develop landing page and user profile

Tasks:

- Design UI using HTML/CSS
- Implement personal data entry
- Set up SQLite database

Sprint 2: Document & Eligibility Engine

Goal: Enable uploads and eligibility logic

Tasks:

- Develop Aadhar and PAN upload module
- Implement eligibility engine in Flask

Sprint 3: Results & Security

Goal: Display results and ensure security

Tasks:

- Design results UI
- Perform security and performance testing
- Add multilingual support (if feasible)

Sprint 4: Review & Deployment

Goal: Finalize and launch system

Tasks:

- Conduct integration testing
- Deploy Flask server
- Provide documentation

Conclusion

The implementation of **Sarkari Sahayata** demonstrates that the Waterfall Model provides the necessary structured approach for secure data handling and clearly defined requirements, while the Agile (Scrum) Model offers the flexibility to adapt to evolving government policies and user feedback. By combining both methodologies, the system achieves a balance between reliability and scalability, resulting in a robust platform that enhances digital accessibility and supports the objectives of **Sustainable Development Goal (SDG) 16**, promoting inclusive and efficient public services.

EXPERIMENT 03

Aim: Analyze the difference between waterfall model and agile model to select the most appropriate process model for the selected problem statement considering economic and financial factors.

CO: Learner will be able to use concepts of software engineering process models to optimize the flow of project development process.

Theory:

Comparison between Waterfall and Agile Model

#	Comparison Factor	Waterfall Model	Agile (Scrum) Model
1	Workflow Structure	Linear and Sequential: Phases must be completed in order (Requirements → Design → Coding → Testing).	Iterative and Incremental: Projects are broken into small cycles called Sprints (1–4 weeks).
2	Requirement Definition	Fixed Upfront: Comprehensive requirements are documented and locked before coding begins.	Evolving: Requirements are maintained in a backlog and updated throughout the project.
3	Customer Involvement	Limited: Input is taken mainly at the beginning and during final review.	Continuous: Stakeholders provide feedback after every sprint.
4	Testing Timing	End-Stage: Testing occurs only after the entire product is built.	Continuous: Testing is integrated into every phase to catch bugs early.
5	Flexibility to Change	Low: Changes are difficult, time-consuming, and expensive.	High: Changes are welcomed even late in development.
6	Product Delivery	Big Bang: Final product is delivered all at once at the end.	Incremental: Working software is delivered frequently.
7	Documentation	Heavy: Detailed documentation at every stage.	Minimal: Focus on working software over documentation.
8	Risk Management	High Risk: Issues are often discovered late.	Low Risk: Regular testing helps early issue detection.
9	Predictability	High: Timeline, budget, and scope are fixed early.	Variable: Project scope and timeline can evolve.
10	Team Structure	Siloed: Teams work in separate specialized roles.	Cross-Functional: Teams collaborate with diverse skills.

Selection & Justification of Model

- Considering the economic and financial factors, the **Agile Model** is selected over the Waterfall Model for the development of the **Sarkari Sahayata Portal**.
- The primary economic driver for choosing Agile is the **Cost of Change**. In a government-centric project, eligibility criteria and scheme rules are subject to frequent legislative updates. In the Waterfall model, a policy change during development would require revisiting earlier phases such as requirements and design, resulting in significant time delays and increased costs. In contrast, Agile treats change as a natural and manageable aspect of development, allowing modifications—such as updates to the *Eligibility Engine*—to be incorporated in upcoming iterations without unnecessary rework or financial loss.
- Furthermore, Agile provides a faster **Return on Investment (ROI)** through the delivery of a **Minimum Viable Product (MVP)**. Instead of waiting for the complete system to be developed, the portal can release essential features, such as basic scheme eligibility checks,

at an early stage. This enables immediate social and economic benefits for users while continuing development in parallel.

- Within the Agile framework, the **Scrum Model** is chosen as the most suitable methodology for this project. Scrum offers a structured yet flexible approach through **time-boxed Sprints**, typically lasting 2–4 weeks. For a system dealing with sensitive user data such as Aadhar and PAN details, Scrum enables dedicated security-focused iterations where aspects like data encryption, secure storage, and access control can be implemented and rigorously tested early in the development cycle.
- This significantly reduces financial risk, as identifying and resolving vulnerabilities during development is far less costly than addressing a major security breach after deployment. Additionally, Scrum's iterative process ensures continuous improvement and quality enhancement throughout the project lifecycle. The role of the **Product Owner** ensures that development priorities remain aligned with the most impactful government schemes, guaranteeing efficient use of resources and maximum public value.

Conclusion

The implementation of **Sarkari Sahayata** demonstrates that the Waterfall Model provides the necessary structured approach for secure data handling and clearly defined requirements, while the Agile (Scrum) Model offers the flexibility required to adapt to evolving government policies and continuous user feedback.

By selecting the Scrum methodology, the project successfully balances financial accountability with technical agility, resulting in a reliable and scalable platform. This approach enhances digital accessibility and supports the objectives of **Sustainable Development Goal (SDG) 16**, promoting inclusive, transparent, and efficient public services.

EXPERIMENT 04

Aim: Write software requirement specification, Work breakdown structure (WBS).

CO: Learner will be able to identify requirements, analyze, plan, schedule and track the progress of the projects.

Theory:

What is SRS?

A Software Requirement Specification (SRS) is a document that clearly describes what a software system should do and the constraints under which it must operate. It serves as a contract between stakeholders, including users, clients, developers, testers, and project managers.

An SRS:

- Defines functional requirements
- Defines non-functional requirements
- Describes external interfaces
- Acts as a baseline for design, development, testing, and maintenance
- Reduces ambiguity and prevents scope creep

IEEE 830–1998 SRS Format

The standard structure includes:

- Title
- Table of Contents
- Introduction
- Overall Description
- Specific Requirements
- Change Management Process
- Document Approval
- Supporting Information (Appendix, Index)

Software Requirement Specification (SRS) for Sarkari Sahayata

1. Introduction

1.1 Purpose

The primary objective of this document is to provide a complete and detailed specification for the Sarkari Sahayata Portal. It is intended for the development team, project supervisors, and government stakeholders to understand the system's functional boundaries and performance expectations.

1.2 Scope

Sarkari Sahayata is an intelligent web platform that matches Indian citizens with relevant government welfare schemes. The system eliminates the information gap by providing a centralized eligibility engine that uses demographic data and identity verification to filter available benefits.

1.3 Definitions, Acronyms, and Abbreviations

- SRS: Software Requirement Specification
- GUI: Graphical User Interface
- Flask: Python-based backend framework
- Eligibility Engine: Core logic module that matches user input with scheme data

1.4 References

- IEEE 830–1998 Standard for Software Requirements Specifications
- Project Problem Statement: Sarkari Sahayata Eligibility Portal

1.5 Overview

This SRS is divided into the following sections: Introduction, Overall Description, Specific Requirements, and External Interface Requirements.

2. Overall Description

2.1 Product Perspective

The portal is a standalone web application operating within a digital governance ecosystem. It uses a SQLite database for storing schemes and HTTPS for secure communication.

2.2 Product Functions

- User Profiling: Captures age, income, category, and state
- Document Upload: Allows secure upload of Aadhar and PAN cards
- Scheme Filtering: Displays eligible schemes based on user data

2.3 User Characteristics

- General Citizens: Basic to intermediate digital literacy
- Administrators: Manage database and system monitoring

2.4 Constraints

- Must comply with Indian data privacy laws for Aadhar data
- Limited to Python 3.x environment using Flask

2.5 Assumptions and Dependencies

- Users have internet access and a web browser
- Accuracy depends on scheme data in CSV/SQLite

3. Specific Requirements

3.1 Functional Requirements

- FR1 (Registration): Users can create profiles with required demographic details
- FR2 (Eligibility Logic): System compares user income with scheme criteria
- FR3 (File Handling): Supports uploads in .jpg, .png, and .pdf formats
- FR4 (Notification): Displays “No Schemes Found” when no matches exist

3.2 Non-Functional Requirements

- Performance: Results displayed within 3 seconds
- Security: AES-256 encryption for stored documents
- Availability: 99.5% uptime
- Usability: Responsive UI for devices with minimum width of 360px

4. External Interface Requirements

4.1 User Interface

- Landing page with search bar and “Check Eligibility” button
- Dashboard built using Bootstrap

4.2 Hardware Interface

- Server: Minimum 2GB RAM, 20GB storage
- Client: Any device with a modern browser

4.3 Software Interface

- Operating System: Linux/Windows
- Database: SQLite 3
- Backend: Python Flask 2.x

4.4 Communication Interface

- Uses TCP/IP protocol
- Data secured via SSL/TLS (HTTPS)

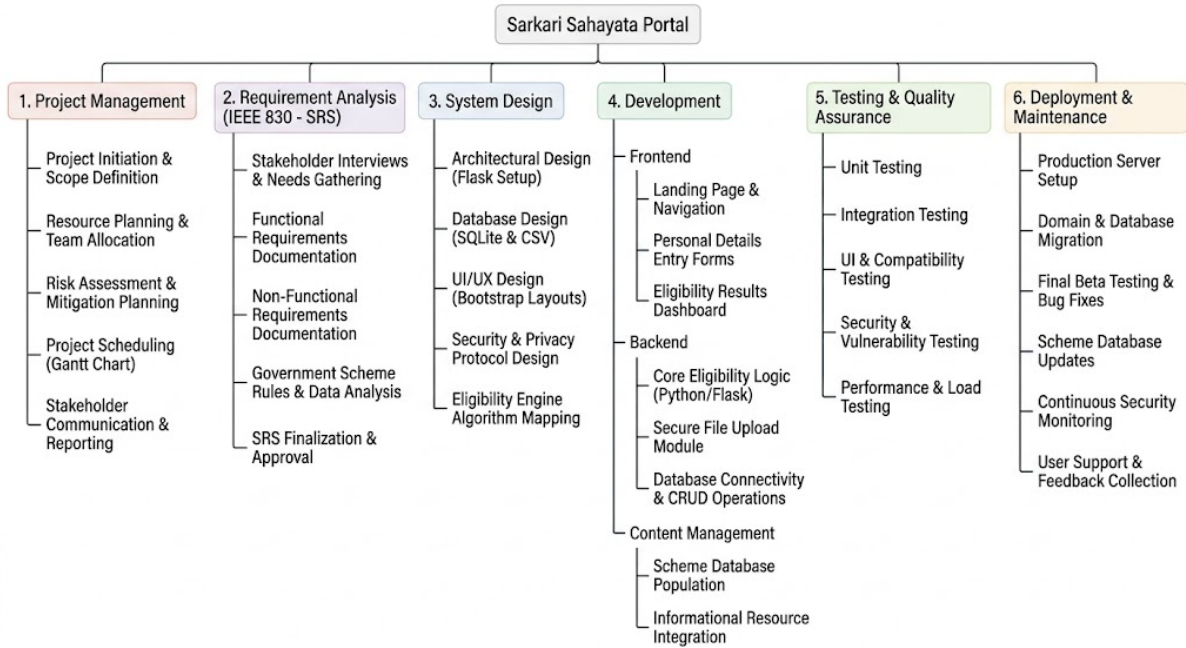
Work Breakdown Structure (WBS)

A Work Breakdown Structure (WBS) is a project management tool that divides a project into smaller, manageable tasks in a hierarchical manner. This simplifies planning, execution, and monitoring.

Need for WBS

- Organizes and defines the complete project scope
- Prevents missing critical tasks
- Ensures systematic execution and control

- Improves scheduling and sequencing
- Helps identify risks early
- Enables accurate estimation of budget and resources



Conclusion

The preparation of the IEEE 830 SRS and a detailed Work Breakdown Structure (WBS) establishes a formal technical agreement and a structured project roadmap.

By clearly defining all functional goals of the Sarkari Sahayata Portal and breaking the project into manageable deliverables, these documents minimize development risks and improve project clarity. They provide a strong foundation for building a transparent, efficient, and citizen-centric eligibility system.

EXPERIMENT 05

Aim: Identify and describe different tools and techniques to prepare a timeline chart. Identify limitations of the tools used to derive a timeline chart. Prepare Gantt chart for the selected problem statement using an appropriate tool.

CO: Learner will be able to identify requirements, analyze, plan, schedule and track the progress of the projects.

Theory:

Timeline Chart

A timeline chart is a visual representation of a sequence of events or tasks displayed in chronological order along a linear axis.

Features:

- Visualizes events on a horizontal or vertical time scale
- Focuses on high-level project milestones and key deadlines
- Displays a linear progression of time from start to finish

Gantt Chart

A Gantt chart is a specialized horizontal bar chart used in project management to illustrate a project schedule.

Features:

- Horizontal bars represent the duration and timing of individual tasks
- Visualizes task dependencies and relationships using arrows or connectors
- Indicates the "Critical Path," showing tasks that directly impact the project end date

Advantages:

- Provides a highly detailed view of the entire project lifecycle
- Helps in identifying resource bottlenecks and scheduling conflicts early
- Facilitates tracking project progress against the original baseline

Disadvantages:

- Can become visually cluttered and overwhelming for very large projects
- Requires frequent manual updates if automation is not available
- Does not inherently represent total cost or budget constraints

Techniques to Create a Timeline Chart

- Work Breakdown Structure (WBS): Breaking the project into smaller, manageable tasks
- Critical Path Method (CPM): Identifying the sequence of tasks that determine project duration
- Program Evaluation and Review Technique (PERT): Using statistical analysis for task completion time
- Milestone Mapping: Identifying key project checkpoints (e.g., SRS Approval)
- Dependency Analysis: Determining task relationships (Finish-to-Start, Start-to-Start, etc.)

Tools to Create Timeline / Gantt Chart

1. Microsoft Project

Description: A specialized project management software used for scheduling, resource allocation, and tracking progress through Gantt charts.

Pros: Offers automated scheduling and professional reporting features.

Cons: High cost and steep learning curve for beginners.

2. Microsoft Excel

Description: A spreadsheet application used to create custom timelines and Gantt charts using bar charts and conditional formatting.

Pros: Easily accessible, no extra cost, and highly flexible.

Cons: Requires manual setup for dependencies and automation.

3. Trello (Timeline View)

Description: A Kanban-based tool that provides a timeline feature to visualize tasks across a calendar.

Pros: Intuitive interface with drag-and-drop functionality.

Cons: Advanced timeline features require a paid version.

4. Lucidchart

Description: A web-based diagramming tool used to manually create timelines and flowcharts.

Pros: Highly customizable with strong visual design and collaboration features.

Cons: Lacks built-in project management logic.

5. Office Timeline

Description: A PowerPoint add-in that converts project data into presentation-ready timeline visuals.

Pros: Ideal for creating professional presentation charts quickly.

Cons: Limited functionality for actual project management tasks.

6. ClickUp

Description: An all-in-one productivity platform with multiple views including timeline and Gantt.

Pros: Highly versatile with automation and multiple workflow views.

Cons: Interface can be overwhelming due to many features.

Justification for Using Microsoft Excel

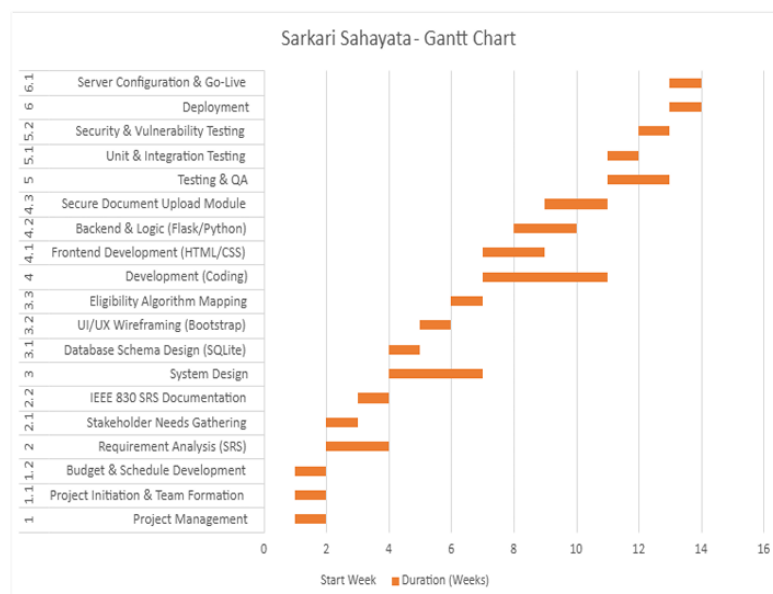
Microsoft Excel was selected as the tool for creating the Sarkari Sahayata Gantt Chart based on the following reasons:

- **Accessibility and Cost:** Excel is widely available in academic environments and does not require additional investment
- **Customization:** Supports conditional formatting to create flexible and customized Gantt charts
- **Ease of Data Integration:** Allows seamless transition from tabular project data to schedule visualization

GANTT CHART

Project Duration: 14 weeks

Project: Sarkari Sahayata



Conclusion

The analysis of project scheduling tools shows that while advanced software provides automation, the choice must align with the project's scale and available resources. The Gantt Chart developed for Sarkari Sahayata serves as a roadmap, ensuring that critical tasks such as eligibility algorithm mapping are prioritized to avoid delays.

Using Excel effectively meets the project requirements by providing a simple yet powerful way to visualize the schedule while maintaining flexibility for iterative development.

EXPERIMENT 06

Aim: Design test cases using black box testing techniques for the selected project, execute the test cases and discuss the results.

CO: Learner will be able to use various software testing methods and strategies to identify bugs.

Theory:

1. State Table Testing

This technique verifies that the system transitions correctly between different operational states based on user inputs.

Sr. No.	Module Name	Test Case Description	Expected Result	Actual Result	Status
1	User Profile	Submit valid demographics and click "Check"	Transition from 'Form Entry' to 'Results'	Transitioned	Pass
2	Document Vault	Click "Upload" for a new Aadhar scan	Transition from 'Idle' to 'Uploading'	Transitioned	Pass
3	Document Vault	Click "Save" after a successful upload	Transition from 'Uploading' to 'Storage Secured'	Transitioned	Pass
4	Authentication	Click "Logout" from the main dashboard	Transition from 'Session Active' to 'Logged Out'	Transitioned	Pass
5	Scheme Detail	Click "View More" on a matched scheme	Transition from 'Summary' to 'Detailed View'	Transitioned	Pass
6	Eligibility	Click "Reset Form" after entering data	Transition from 'Data Entered' to 'Empty Form'	Transitioned	Pass

2. Equivalence Partitioning

Input data is divided into valid and invalid partitions to ensure broad coverage with fewer test cases.

Sr. No.	Module Name	Test Case Description	Expected Result	Actual Result	Status
1	Eligibility Engine	Input Age: 30 (Valid Partition: 1–120)	Processing successful	Processed	Pass
2	Eligibility Engine	Input Age: "Thirty" (Invalid Partition: String)	Error: "Numeric input required"	Error shown	Pass
3	File Upload	Upload .pdf or .jpg (Valid Partition)	Upload successful	Uploaded	Pass
4	File Upload	Upload .exe or .sh (Invalid Partition)	Error: "Unsupported file type"	Error shown	Pass
5	Income Field	Input: -5000 (Invalid Partition: Negative)	Error: "Income cannot be negative"	Error shown	Pass
6	Income Field	Input: 12,00,000 (Valid Partition: Numeric)	Processing successful	Processed	Pass

3. Error Guessing

This method relies on predicting common user mistakes or system loopholes.

Sr. No.	Module Name	Test Case Description	Expected Result	Actual Result	Status
1	Registration	Submit form with all fields blank	System blocks submission and highlights fields	Blocked	Pass

Sr. No.	Module Name	Test Case Description	Expected Result	Actual Result	Status
2	Search	Enter special characters (@#\$\$%^)	No results or "Invalid search"	No results	Pass
3	Eligibility	Enter income ₹1	Matches poverty-aligned schemes	Processed	Pass
4	Document Vault	Rapidly click "Upload" multiple times	Only first click processed	Single upload	Pass
5	Login	Enter valid email with trailing spaces	System trims spaces and logs in	Logged in	Pass
6	Document Vault	Upload same document twice	Prompt to replace or keep both	Prompt shown	Pass

4. Boundary Value Analysis (BVA)

BVA focuses on values at the extreme ends of input ranges where logic errors are most frequent.

Sr. No.	Module Name	Test Case Description	Expected Result	Actual Result	Status
1	Age Field	Input Age: 0 (Lower Boundary)	Accepted (Infant-targeted schemes)	Accepted	Pass
2	Age Field	Input Age: 120 (Upper Boundary)	Accepted (Senior citizen schemes)	Accepted	Pass
3	File Size	Upload file of exactly 10MB (Limit)	Upload successful	Uploaded	Pass
4	File Size	Upload file of 10.1MB (Just over limit)	Error: "File size exceeds limit"	Error shown	Pass
5	Income Field	Input exactly ₹2,50,000 (Threshold)	Correct threshold logic applied	Correct match	Pass
6	Income Field	Input exactly ₹0 (Minimum Boundary)	Maximum-benefit schemes identified	Correct match	Pass

Conclusion

The design and execution of these 24 test cases using Black Box testing techniques ensure that the Sarkari Sahayata portal functions correctly under various scenarios. The system demonstrated stability in state transitions, correctly handled boundary conditions, and effectively filtered invalid data inputs, confirming its readiness for advanced testing phases.

EXPERIMENT 07

Aim: Design test cases using path testing technique and execute test cases for the same selected problem statement to identify, analyze and fix the logical errors.

CO: Learner will be able to use various software testing methods and strategies to identify bugs.

Theory:

Path Testing

Path testing is a white-box software testing technique in which test cases are designed based on the control flow of a program. In this method, the source code is analyzed to identify all possible execution paths from the entry point to the exit point of a program.

White-box testing, also known as clear-box testing, glass-box testing, or structured testing, is a methodology where the tester has complete knowledge of the internal structure, logic, and code of the application.

Control Flow Graph (CFG)

A Control Flow Graph (CFG) is a directed graph used to represent all possible execution paths in a program.

- Nodes represent basic blocks (a sequence of statements with no branching)
- Edges represent the flow of control between these blocks
- Decision nodes are nodes with more than one outgoing edge, representing conditional branching

Types of Path Testing Coverage

- Statement Coverage: Ensures every statement in the code is executed at least once
- Branch Coverage: Ensures every possible branch (true/false) is executed
- Path Coverage: Ensures all possible execution paths are tested
- Condition Coverage: Tests all possible outcomes of individual conditions
- Loop Coverage: Ensures loops are tested for different iterations (zero, one, multiple)

Advantages

- Provides thorough code coverage
- Helps in early detection of bugs
- Reveals hidden or complex execution paths
- Assists in code optimization
- Useful for identifying security vulnerabilities
- Forms the basis for cyclomatic complexity calculation

Disadvantages

- Not scalable for large and complex systems
- Requires access to source code
- Can be time-consuming and expensive
- Cannot detect missing functionality
- Does not evaluate user experience

For SarkariSahayata, path testing was applied to five critical user flows: User Registration with OTP, Login, Eligibility Checking, Scheme Browsing, and Chatbot Interaction.

Test Case 1: User Registration with OTP Verification

Details

Test Case ID: TC001

Test Description: Path testing for user registration and OTP email verification

Precondition: User is on the registration page (/auth/register). No account exists with the test email

Test Steps

- Navigate to /auth/register
- Enter valid username, email, and password

- Click "Send OTP & Register"
- Check email inbox for OTP
- Enter OTP on verification page
- Submit form

Input Data

Field	Value
Username	testuser01
Email	testuser01@gmail.com
Password	Test@1234
OTP	482917

Expected Result

Account is created, OTP is verified, and user is redirected to login page

Code Snippet (auth.py)

```
if not username:
    error = 'Username is required.'
elif not password:
    error = 'Password is required.'
elif not emails:
    error = 'Email is required.'
else:
    error = validate_password(password, username)
cur.execute("SELECT id FROM user WHERE username = %s OR email = %s", (username, emails))
duration = 600
if (record and record['otp'] == entered_otp and (time.time() - float(record['created'])) < duration):
    cur.execute("UPDATE user SET is_registered = 1 WHERE id = %s", (user['id'],))
```

Paths Table

Path	Flow
1	Enter fields → All fields valid → No existing user → OTP sent → Correct OTP entered → Not expired (< 600s) → is_registered = 1 → Redirect to /auth/login
2	Enter fields → All fields valid → Existing user found → Error: "Username or email already taken"
3	Enter fields → Password empty → elif not password → Validation error: "Password is required"
4	Enter fields → All fields valid → OTP sent → Wrong OTP entered → Condition fails → Error: "Invalid OTP"
5	Enter fields → All fields valid → OTP sent → Correct OTP → Expired (time > 600s) → Condition fails → Error: "Invalid or expired OTP"
6	Enter fields → OTP requested repeatedly → Rate limit triggered → HTTP 429: "Too many requests"

Results

Path	Path Description	Expected Outcome	Actual Result	Status
1	Valid input + correct OTP	Registration successful	Registration successful	Pass
2	Existing email	Error message	Error displayed	Pass
3	Empty password	Validation error	Submission blocked	Pass
4	Wrong OTP	Error message	Error shown	Pass
5	Expired OTP	Error message	Expired OTP rejected	Pass
6	Excess OTP requests	Rate limit enforced	Rate limit applied	Pass

Test Case 2: User Login Process

Details

Test Case ID: TC002

Test Description: Path testing for login system

Precondition: User account exists and is verified

Test Steps

- Navigate to /auth/login
- Enter username and password
- Click "Sign In Securely"

Input Data

Field	Value
Username	testuser01
Password	Test@1234

Expected Result

User is authenticated and redirected to homepage

Code Snippet (auth.py)

```
cur.execute("SELECT * FROM user WHERE username = %s", (username,))
user = cur.fetchone()
if user is None:
    error = 'Incorrect username.'
elif not check_password_hash(user['password'], password):
    error = 'Incorrect password.'
elif user['is_registered'] == 0:
    error = 'Email not verified.'
if error is None:
    session['user_id'] = user['id']
```

Paths Table

Path	Flow
1	Enter username → User found → Password matches → is_registered = 1 → error is None → session created → Redirect to /
2	Enter username → User found → Password does not match → Error: "Incorrect password"
3	Enter username → User not found → Error: "Incorrect username"
4	Enter username → User found → Password matches → is_registered = 0 → Error: "Email not verified"
5	Enter username → Empty field → Client-side validation → Submission blocked
6	Access /eligibility → g.user is None → login_required decorator → Redirect to /auth/login

Results

Path	Path Description	Expected Outcome	Actual Result	Status
1	Correct credentials	Login success	Login successful	Pass
2	Wrong password	Error	Error shown	Pass
3	Invalid username	Error	Error shown	Pass
4	Unverified account	OTP prompt	Verification prompt	Pass
5	Empty field	Validation error	Submission blocked	Pass
6	Unauthorized access	Redirect	Redirect enforced	Pass

Test Case 3: Eligibility Checker

Details

Test Case ID: TC003

Test Description: Path testing for eligibility engine

Precondition: User is logged in

Test Steps

- Navigate to /eligibility
- System processes user data
- View results

Input Data

Field	Value
Age	24
Income	₹1,20,000
Gender	Female
Caste	SC
State	Maharashtra
Occupation	Student

Expected Result

Eligible schemes are displayed

Code Snippet (home.py)

```
if not g.user:
    return redirect(url_for("auth.login"))
for scheme in schemes:
    if scheme["max_income"] and income > scheme["max_income"]:
        is_eligible = False
```

Paths Table

Path	Flow
1	g.user exists → user_details found → Loop schemes → All criteria match → Eligible list populated → Display results
2	g.user exists → user_details found → Loop schemes → No scheme matches → Eligible list empty → Show "No schemes found" message
3	g.user exists → user_details not found → Redirect to /add-details
4	g.user exists → user_details found → High income → scheme["max_income"] check fails → is_eligible = False → Only NULL-income schemes shown
5	g.user is None → not g.user → Redirect to /auth/login
6	g.user exists → user_details found → Scheme has NULL criteria → Condition skipped → Scheme included for all users

Results

Path	Path Description	Expected Outcome	Actual Result	Status
1	Matching schemes	Display results	Schemes displayed	Pass
2	No match	Show message	Message shown	Pass
3	Missing data	Redirect	Redirect enforced	Pass
4	High income	Limited schemes	Correct filtering	Pass
5	Not logged in	Redirect	Redirect enforced	Pass
6	NULL criteria	Included	Included correctly	Pass

Test Case 4: Scheme Browsing by Category

Details

Test Case ID: TC004

Test Description: Path testing for category browsing

Test Steps

- Select category
- View schemes
- Open scheme details

Input Data

Field	Value
Category	Health
Scheme	Ayushman Bharat

Expected Result

Category and scheme details displayed

Code Snippet

```
cursor.execute("""
```

```
SELECT * FROM schemes
```

```
WHERE LOWER(category) = LOWER(%s)
```

```
""", (category_name,))
```

Paths Table

Path	Flow
1	Select category → DB query executes → Schemes found (fetchall not empty) → Render category page with schemes
2	Select category → DB query executes → No schemes found (fetchall empty) → Render "No schemes available" message
3	Click scheme → Valid scheme_id → fetchone returns scheme → Render scheme_detail.html
4	Click scheme → Invalid scheme_id → fetchone returns None → abort(404)
5	Select language → Select category → LOWER() query executes → Translated scheme titles and descriptions rendered

Results

Path	Path Description	Expected Outcome	Actual Result	Status
1	Valid category	Schemes displayed	Schemes loaded	Pass
2	No schemes	Message shown	Message shown	Pass
3	Valid ID	Details shown	Details shown	Pass
4	Invalid ID	404 error	404 handled	Pass
5	Language change	Translated content	Translation works	Pass

Test Case 5: AI Chatbot Interaction

Details

Test Case ID: TC005

Test Description: Path testing for chatbot

Test Steps

- Open chatbot
- Enter query
- Submit

Input Data

Query Type	Input
English	What are the health schemes?
Hindi	स्वास्थ्य योजनाएँ दिखाओ
Empty	(blank)

Expected Result

Chatbot provides correct response

Code Snippet

```
def contains_any(text, words):
    for word in words:
        if word.lower() in text:
            return True
```

Paths Table

Path	Flow
1	Click chat button → iframe is hidden → toggle → iframe becomes visible → Chat opens
2	Enter English query → contains_any loops keywords → Match found → DB query → Response returned in English
3	Enter Hindi query → contains_any loops Hindi keywords → Match found → Response returned in Hindi
4	Submit empty input → contains_any finds no match → No DB call → Default reply shown
5	Click chat button → iframe is visible → toggle → iframe becomes hidden → Chat closes
6	Enter unknown query → contains_any loops all keywords → No match found → Fallback message displayed

Results

Path	Path Description	Expected Outcome	Actual Result	Status
1	Open chat	Chat opens	Opened	Pass
2	English query	Correct response	Correct response	Pass
3	Hindi query	Hindi response	Hindi response	Pass
4	Empty input	Default message	Default shown	Pass
5	Toggle chat	Close	Closed	Pass
6	Unknown query	Fallback	Fallback shown	Pass

Conclusion

Path Testing proved to be an effective technique for verifying the logical correctness and robustness of the SarkariSahayata portal. By systematically identifying and executing all possible execution paths across five critical modules, the testing process confirmed that the system handles both valid and invalid scenarios correctly.

The testing validated key functionalities such as OTP-based registration with expiry and rate-limiting logic, the eligibility engine with NULL criteria handling, and the chatbot's multilingual response system. It also ensured that all protected routes properly restrict unauthorized access using authentication checks.

All 29 paths across the five test cases were successfully executed, indicating that the application is free from major logical errors. The system is now ready for further testing phases such as performance and security testing.

EXPERIMENT 08

Aim: Study and Categorize Various Testing Tools and Implement One Open-Source Testing Tool

CO: Learner will be able to illustrate use of automation tools for software testing.

Theory:

A testing tool is a software application used to automate, manage, execute, and monitor software testing activities to ensure that the system works correctly and meets specified requirements.

Software testing tools are an essential part of modern software development. They help testers and developers verify the correctness, reliability, and performance of applications. These tools reduce manual effort, improve testing speed, and enhance accuracy. They are used at different stages of the software development lifecycle such as unit testing, integration testing, system testing, and acceptance testing.

1. Unit Testing Tools

Definition

Unit testing tools are used to test individual units or components of a program in isolation.

Overview

These tools are mainly used by developers to ensure that each function or module works correctly before integration.

Uses

- Testing small code components
- Debugging code early
- Improving code quality

Advantages

- Early bug detection
- Easy debugging
- Faster execution

Disadvantages

- Limited to small units
- Requires coding knowledge
- Cannot detect integration issues

Examples

JUnit

- Java-based unit testing tool
- Uses annotations like @Test
- Widely used in development

NUnit

- Used for .NET applications
- Supports assertions
- Easy execution

PyTest

- Simple syntax
- Supports fixtures
- Highly extensible

2. Integration Testing Tools

Definition

Integration testing tools test the interaction between different modules or systems.

Overview

After unit testing, modules are combined and tested to ensure proper communication.

Uses

- Testing APIs
- Checking data flow
- Verifying module interaction

Advantages

- Detects interface errors
- Ensures system compatibility
- Validates data exchange

Disadvantages

- Complex setup
- Difficult debugging
- Time-consuming

Examples

Postman

- Used for API testing
- Easy request execution
- Supports automation

SoapUI

- SOAP and REST support
- Functional and load testing
- Script support

Cucumber

- Uses simple English syntax
- Supports BDD
- Improves collaboration

3. Functional Testing Tools

Definition

Functional testing tools verify whether the system performs according to specified requirements.

Overview

These tools test application features such as login, search, and forms.

Uses

- Feature validation
- User workflow testing
- UI interaction

Advantages

- Ensures requirement compliance
- Detects functional defects
- Supports automation

Disadvantages

- Does not test performance
- Requires proper test cases
- Can be time-consuming

Examples

Selenium

- Automates web browsers
- Supports multiple languages
- Cross-browser testing

Cypress

- Fast execution

- Real-time debugging
- Modern tool

Appium

- Mobile testing
- Supports Android and iOS
- Cross-platform

Katalon Studio

- Easy to use
- Supports multiple testing types
- Built-in reports

4. Performance Testing Tools

Definition

Performance testing tools evaluate system performance under load.

Overview

These tools measure speed, responsiveness, and stability.

Uses

- Load testing
- Stress testing
- Performance monitoring

Advantages

- Identifies bottlenecks
- Improves scalability
- Ensures reliability

Disadvantages

- Complex configuration
- Requires expertise
- Resource intensive

Examples

Apache JMeter

- Open-source
- GUI interface
- Supports multiple protocols

LoadRunner

- Enterprise-level tool
- Simulates many users
- Advanced reporting

Gatling

- High performance
- Code-based testing
- Detailed analytics

5. Security Testing Tools

Definition

Security testing tools identify vulnerabilities and threats in a system.

Overview

They ensure that applications are protected against attacks.

Uses

- Vulnerability detection
- Penetration testing

- Risk assessment

Advantages

- Improves security
- Prevents data breaches
- Protects users

Disadvantages

- Complex usage
- False positives
- Requires expertise

Examples

OWASP ZAP

- Open-source
- Easy to use
- Detects vulnerabilities

Burp Suite

- Professional tool
- Advanced features
- Used by experts

Wireshark

- Network analysis
- Packet capture
- Deep inspection

6. Regression Testing Tools

Definition

Regression testing tools ensure that new changes do not affect existing features.

Overview

Used after updates or bug fixes.

Uses

- Re-testing
- Stability verification
- Change validation

Advantages

- Maintains stability
- Detects new bugs
- Saves time

Disadvantages

- Repetitive process
- Maintenance required
- Time-consuming

Examples

Selenium

- Automates regression testing
- Reusable scripts
- Cross-browser

TestComplete

- Supports multiple platforms
- Easy automation
- Scriptless option

Ranorex

- GUI testing
- Easy integration
- Good reporting

7. UI Testing Tools

Definition

UI testing tools test the user interface of applications.

Overview

These tools focus on visual and interactive elements.

Uses

- Button testing
- Form validation
- Layout checking

Advantages

- Improves user experience
- Detects UI bugs
- Automates UI tasks

Disadvantages

- Sensitive to UI changes
- Requires maintenance
- Slower than unit testing

Examples

Selenium

- UI automation
- Cross-browser
- Flexible

TestCafe

- No WebDriver required
- Easy setup
- Fast execution

Cypress

- Modern UI testing
- Real-time updates
- Debugging support

8. Test Management Tools

Definition

Test management tools help manage test cases, plans, and execution.

Overview

Used by QA teams to organize testing activities.

Uses

- Test planning
- Test tracking
- Reporting

Advantages

- Organized workflow
- Better collaboration
- Detailed reports

Disadvantages

- Costly tools
- Requires training
- Setup complexity

Examples

TestRail

- Manages test cases
- Generates reports
- Tracks results

PractiTest

- Centralized system
- Real-time updates
- Integration support

Xray

- Works with Jira
- Agile support
- Defect tracking

Implementation of Open-Source Testing Tool (Selenium)

Selenium is one of the most popular open-source testing tools used for automating web browsers. It is widely used for functional testing of web applications. Selenium supports multiple programming languages such as Python, Java, and C#, and works across different browsers like Chrome, Firefox, and Edge.

Steps to Execute Selenium Script

1. Install Python
Download and install Python from the official website and ensure it is added to the system PATH
2. Install Selenium
Open terminal or command prompt and run
`pip install selenium`
3. Download ChromeDriver
Download ChromeDriver that matches your Chrome browser version from the official website
4. Extract ChromeDriver Path
Extract the downloaded file and note the file path
5. Update Path in Script
Provide the ChromeDriver path in the Selenium service configuration
6. Create Python File
Save the script file as `test.py`
7. Execute the Script
Run the file using the command
`python test.py`
8. Execution Output
Chrome browser will open automatically, perform the defined actions, and close on its own

Benefits of Using Selenium

- Cross-browser support (Chrome, Firefox, Edge)
- Supports multiple programming languages
- Open-source and free to use
- Easy integration with other tools
- Supports automation for web applications

```

Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\sidra>pip install selenium
Collecting selenium
  Downloading selenium-4.43.0-py3-none-any.whl.metadata (7.5 kB)
Collecting certifi<=2026.1.4 (from selenium)
  Downloading certifi-2026.2.25-py3-none-any.whl.metadata (2.5 kB)
Collecting trio<1.0,>=0.31.0 (from selenium)
  Downloading trio-0.33.0-py3-none-any.whl.metadata (8.5 kB)
Collecting trio-websocket<1.0,>=0.12.2 (from selenium)
  Downloading trio_websocket-0.12.2-py3-none-any.whl.metadata (5.1 kB)
Requirement already satisfied: typing_extensions<5.0,>=4.15.0 in c:\users\sidra\appdata\local\packages\pythonsoftwarefou
ndation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from selenium) (4.15.0)
Collecting urllib3<3.0,>=2.6.3 (from urllib3[socks]<3.0,>=2.6.3->selenium)
  Downloading urllib3-2.6.3-py3-none-any.whl.metadata (6.9 kB)
Collecting websocket-client<2.0,>=1.8.0 (from selenium)
  Downloading websocket_client-1.9.0-py3-none-any.whl.metadata (8.3 kB)
Requirement already satisfied: attrs<=23.2.0 in c:\users\sidra\appdata\local\packages\pythonsoftwarefoundation.python.3.
11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from trio<1.0,>=0.31.0->selenium) (25.4.0)
Collecting sortedcontainers (from trio<1.0,>=0.31.0->selenium)
  Downloading sortedcontainers-2.4.0-py2.py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: idna in c:\users\sidra\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2
kfra8p0\localcache\local-packages\python311\site-packages (from trio<1.0,>=0.31.0->selenium) (3.11)
Collecting outcome (from trio<1.0,>=0.31.0->selenium)
  Downloading outcome-1.3.0.post0-py2.py3-none-any.whl.metadata (2.6 kB)
Requirement already satisfied: sniffio<=1.3.0 in c:\users\sidra\appdata\local\packages\pythonsoftwarefoundation.python.3
.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from trio<1.0,>=0.31.0->selenium) (1.3.1)
Requirement already satisfied: cffi<=1.14 in c:\users\sidra\appdata\local\packages\pythonsoftwarefoundation.python.3.11_
qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from trio<1.0,>=0.31.0->selenium) (2.0.0)

```

```

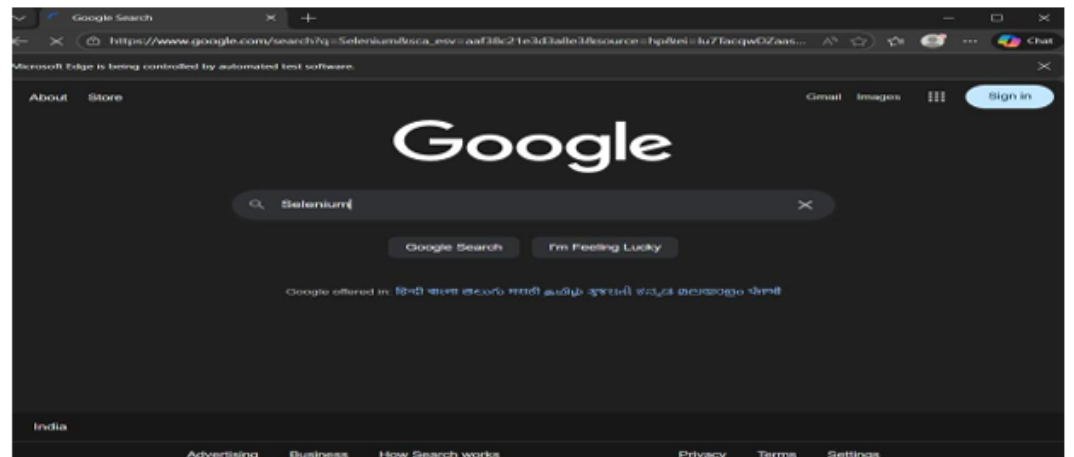
Attempting uninstall: urllib3
  Found existing installation: urllib3 2.5.0
  Uninstalling urllib3-2.5.0:
    Successfully uninstalled urllib3-2.5.0
Attempting uninstall: certifi
  Found existing installation: certifi 2025.10.5
  Uninstalling certifi-2025.10.5:
    Successfully uninstalled certifi-2025.10.5
Successfully installed certifi-2026.2.25 outcome-1.3.0.post0 pysocks-1.7.1 selenium-4.43.0 sortedcontainers-2.4.0 trio-0.33.0 trio-websocket-0.12.2
urllib3-2.6.3 websocket-client-1.9.0 wsproto-1.3.2

[notice] A new release of pip is available: 20.3 -> 26.0.1
[notice] To update, run: C:\Users\sidra\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\python.exe -m pip ins
tall --upgrade pip

C:\Users\sidra>python test.py

```

Name	Date modified	Type	Size
chromedriver	4/15/2026 7:07 PM	Application	21,009 KB
test	4/15/2026 7:11 PM	Python File	1 KB



Code:

```
# Import webdriver to control the browser
from selenium import webdriver

# Import By to locate elements on the webpage
from selenium.webdriver.common.by import By

# Import Keys to simulate keyboard actions
from selenium.webdriver.common.keys import Keys

# Import Service to specify ChromeDriver path
from selenium.webdriver.chrome.service import Service

# Import time to pause execution
import time

# Set the path of ChromeDriver executable
service = Service(r'C:\Users\harsh\Downloads\chromedriver-win64\chromedriver.exe')

# Launch Chrome browser using WebDriver
driver = webdriver.Chrome(service=service)

# Open Google website
driver.get("https://www.google.com")

# Locate the search box using name attribute 'q'
search_box = driver.find_element(By.NAME, 'q')

# Type 'Selenium' in the search box
search_box.send_keys('Selenium')

# Press Enter key to perform search
search_box.send_keys(Keys.RETURN)

# Wait for 5 seconds to see the result
time.sleep(5)

# Close the browser
driver.quit()
```

Conclusion:

Testing tools are categorized based on the type of testing they perform. Selenium is an excellent open-source testing tool for automating web application testing. It is easy to implement and highly flexible. The above example demonstrates how Selenium can be used with Python to automate a simple search operation on a web browser.

EXPERIMENT 09

Aim: Study and implementation of any one bug tracking tool.

CO: Learner will be able to illustrate use of automation tools for software testing.

Theory:

Introduction

Bug tracking tools are software applications used to identify, report, manage, and track bugs or defects in software projects. They help development and testing teams monitor issues from detection to resolution. These tools improve workflow, communication, and overall product quality.

Why Bug Tracking Tools are Needed

1. To record and manage software defects properly
2. To assign bugs to responsible team members
3. To track bug status such as open, in progress, resolved, and closed
4. To improve communication between developers, testers, and managers
5. To ensure software is reliable and error-free

Popular Bug Tracking Tools

1. Jira
2. Bugzilla
3. Redmine
4. Trello
5. GitHub

Jira is a widely used project management and issue tracking tool developed by Atlassian. It offers a variety of automation features that streamline the software testing process. By integrating automation into the testing pipeline, teams can automate repetitive tasks, improve efficiency, and ensure better traceability of defects and test cases.

Automation Using Jira in Software Testing

1. Automated Test Case Creation

In Agile development, each feature or user story requires corresponding test cases. Jira automation helps streamline this process by automatically creating test cases when a new user story or feature is added.

Example Automation Rule

Trigger: When a new user story is created in Jira with a specific label such as "Test"

Action: Automatically create a test case in an external testing tool like Zephyr or Xray, or generate a test subtask within Jira

Benefits

1. Saves time by automatically generating test cases
2. Ensures all new features have corresponding test cases
3. Provides seamless linking between stories and test cases
4. Reduces chances of missing test cases

Disadvantages

1. Incorrect rules may create unnecessary test cases
2. Complex projects require careful setup

2. Automated Test Execution Tracking

During testing, automated tools execute test scripts and produce results such as pass, fail, or skipped. Jira integrates with tools like Jenkins, Selenium, and TestNG to capture these results and update records.

Example Automation Rule

Trigger: When a Jenkins job completes execution of automated tests

Action: Jira updates issue status or attaches test results to the corresponding issue

Benefits

1. Eliminates manual updates from automated execution
2. Ensures issue status reflects actual testing progress
3. Provides traceability of test results within Jira

Disadvantages

1. Initial integration setup may be difficult
2. Failures in connected tools may affect updates

3. Automatic Test Status Update

As testing progresses, the status of bugs and test cases keeps changing. Jira automation updates statuses automatically based on events like test completion or approval.

Example Automation Rule

Trigger: When a bug or test case is updated or test execution is completed

Action: Automatically update the test or defect status

Benefits

1. Keeps test status accurate and up to date
2. Improves communication across teams
3. Helps track testing progress and defect resolution

Disadvantages

1. Incorrect conditions may set wrong status

4. Automating Defect Creation from Failed Tests

When an automated test case fails, it indicates a defect. Jira can automatically create a defect issue with all necessary details.

Example Automation Rule

Trigger: Test execution result is failed

Action: Automatically create a defect issue with details like failed steps, screenshots, logs, and test case ID

Benefits

1. Reduces manual effort in defect reporting
2. Ensures no failures go unnoticed
3. Enables faster issue resolution

Disadvantages

1. Duplicate bugs may be created
2. Requires filtering and validation rules

5. Automating Test Reporting

Managers and stakeholders require regular reports showing testing progress, pass percentage, and defect count. Jira automation generates and sends reports automatically.

Example Automation Rule

Trigger: End of sprint or test cycle

Action: Generate and send test summary report to stakeholders

Benefits

1. Saves time by eliminating manual report creation
2. Ensures consistency in reporting
3. Provides real-time insights into testing progress

Disadvantages

1. Poor report design may hide useful insights

6. Automatically Closing Issues After Test Pass

After bugs are fixed and verified, issues must be closed. Jira automation ensures issues are closed when all required conditions are satisfied.

Example Automation Rule

Trigger: When a test case passes or issue is marked resolved

Action: Automatically transition issues to "Closed" or "Done"

Benefits

1. Ensures only completed tasks are closed
2. Reduces administrative effort
3. Maintains accurate project status
4. Ensures smooth workflow across development and testing

Disadvantages

1. Incorrect rules may close unresolved issues

7. Automatic Test Environment Setup

Testing often requires environments such as staging servers, databases, or cloud infrastructure. Jira automation helps trigger environment setup when testing begins.

Example Automation Rule

Trigger: When test cycle is marked ready or sprint begins

Action: Notify DevOps team or trigger API to deploy testing environment

Benefits

1. Simplifies environment management
2. Reduces manual intervention
3. Ensures environments are ready for testing
4. Saves setup time and avoids delays

Disadvantages

1. Requires DevOps integration
2. Environment setup failure may delay testing

Jira Bug Tracking Report – SarkariSahayata

Project: SarkariSahayata

Module: auth.py

Sprint: 1

Bug Details

BUG-001

Summary: Password validation allows weak passwords due to incomplete special character check

Component: Authentication / Registration

Priority: Medium

Status: To Do

Reporter: Agasthi Doshi

Assignee: Gautam Kishnani

Description:

The validate_password() function only checks for !, @, # using the regex [!@#]. Passwords like Password1# pass, but Password1\$ should also pass. This limits valid passwords and weakens validation.

Steps to Reproduce

1. Go to /auth/register
2. Enter username: testuser
3. Enter password: Password1!
4. Enter password: Pass1word
5. Enter password: Password\$1

Expected Result

All standard special characters should be accepted

Actual Result

Only !, @, # are accepted

Fix

Change regex to r"[!@#\$%^&*(),.?":{}|<>]"

BUG-002

Summary: NameError in create_otp() due to undefined variable user_id

Component: Authentication / OTP

Priority: Critical

Status: In Progress

Reporter: Shushant Kharate

Assignee: Agasthi Doshi

Description:

The create_otp(id) function contains a duplicate cur.execute() that references user_id, which is not defined. This causes a NameError during registration and shows an incorrect error message.

Steps to Reproduce

1. Go to /auth/register
2. Enter valid details
3. Submit form
4. Observe incorrect error

Expected Result

OTP is generated and user is redirected

Actual Result

Error shown even for new users

Fix

Remove duplicate cur.execute() block

BUG-003

Summary: OTP generated is 4 digits instead of 6 digits

Component: Authentication / OTP

Priority: High

Status: To Do

Reporter: Harshit Singh

Assignee: Shushant Kharate

Description:

OTP uses randint(1000, 9999) which generates 4-digit values, reducing security.

Steps to Reproduce

1. Register user
2. Check OTP

Expected Result

6-digit OTP

Actual Result

4-digit OTP

Fix

Change to randint(100000, 999999)

BUG-004

Summary: Broad exception block masks real errors

Component: Authentication / Registration

Priority: High

Status: To Do

Reporter: Agasthi Doshi

Assignee: Gautam Kishnani

Description:

All errors show the same message "User already registered", making debugging difficult.

Steps to Reproduce

1. Disable email server
2. Register user

Expected Result

Specific error messages

Actual Result

Generic error shown

Fix

Split try-except blocks

BUG-005

Summary: No duplicate email check before registration

Component: Authentication / Registration

Priority: Medium

Status: To Do

Reporter: Gautam Kishnani

Assignee: Harshit Singh

Description:

System relies on database constraints instead of checking email explicitly.

Steps to Reproduce

1. Register with email
2. Register again with same email

Expected Result

Proper email duplicate message

Actual Result

Incorrect username-based error

BUG-006

Summary: No rate limiting on login route

Component: Authentication / Login

Priority: High

Status: In Progress

Reporter: Harshit Singh

Assignee: Shushant Kharate

Description:

Unlimited login attempts allow brute-force attacks.

Steps to Reproduce

1. Send multiple login requests
2. Observe no restriction

Expected Result

Limit after 5 attempts

Actual Result

Unlimited attempts allowed

Fix

Add rate limiter

BUG-007

Summary: OTP expires in 60 seconds instead of 10 minutes

Component: Authentication / OTP Verification

Priority: Critical

Status: In Progress

Reporter: Shushant Kharate

Assignee: Agasthi Doshi

Description:

OTP expires too quickly, causing verification failures.

Steps to Reproduce

1. Register user
2. Wait for OTP
3. Enter OTP

Expected Result

Valid for 10 minutes

Actual Result

Expires in 60 seconds

Fix

Change duration to 600 seconds

BUG-008

Summary: OTP records not deleted after verification

Component: Authentication / OTP Verification

Priority: Medium

Status: To Do

Reporter: Gautam Kishnani

Assignee: Harshit Singh

Description:

OTP entries remain in database after verification, creating a security risk.

Steps to Reproduce

1. Register and verify user
2. Check database

Expected Result

OTP deleted

Actual Result

OTP remains

Fix

Delete OTP after verification

Bug Summary Table

Bug ID	Summary	Priority	Status
BUG-001	Incomplete password validation	Medium	To Do
BUG-002	NameError in OTP function	Critical	In Progress
BUG-003	OTP is 4 digits	High	To Do
BUG-004	Broad exception masking errors	High	To Do
BUG-005	No duplicate email check	Medium	To Do
BUG-006	No rate limiting on login	High	In Progress
BUG-007	OTP expiry too short	Critical	In Progress
BUG-008	OTP not deleted after verification	Medium	To Do

Conclusion

Leveraging Jira automation tools can significantly enhance the efficiency, effectiveness, and traceability of the software testing process. It helps streamline workflows, reduce manual effort, and improve communication across teams, leading to faster and more reliable software delivery.

EXPERIMENT 10

Aim: Identify Quality Attributes and Relationships for the Selected Problem Statement.

CO: Learner will be able to apply software quality assurance factors to identify quality attributes and their relationships.

Theory:

Quality attributes are non-functional characteristics that determine how well a software system performs, responds, and sustains itself in real-world conditions. They focus not only on what the system does, but also on how effectively and securely it delivers those functions.

For the project Sarkari Sahayata, quality attributes are highly important because the platform helps citizens find government schemes, verify eligibility, register securely, and access scheme details in multiple languages. Since the system is designed for students, farmers, women, senior citizens, and unemployed individuals, it must be reliable, secure, easy to use, and accessible.

Below are the key quality attributes and their relationships.

1. Performance

Definition

Performance refers to the speed and responsiveness of the system while processing user actions such as login, searching schemes, translating content, and checking eligibility.

Importance

Users should receive results quickly. Slow response reduces trust.

Relationship with Other Attributes

Usability: Fast response improves experience

Scalability: Performance must remain stable with more users

Customer Satisfaction: Quick results create positive impressions

Example

Eligible schemes should be displayed within a few seconds

2. Security

Definition

Security protects user accounts, credentials, and system data from unauthorized access.

Importance

Sensitive user data like email and personal details must be protected.

Relationship with Other Attributes

Reliability: Secure systems are dependable

Maintainability: Security updates require maintainable code

Usability: Security should not make usage difficult

Example

OTP verification, password hashing, session management

3. Reliability

Definition

Reliability is the ability to consistently provide correct outputs without failure.

Importance

Users depend on correct scheme recommendations.

Relationship with Other Attributes

Availability: Reliable systems stay operational

Maintainability: Bugs can be fixed easily

Performance: Stable systems provide consistent speed

Example

Eligibility checker should always provide accurate results

4. Usability

Definition

Usability refers to how easy the system is to use.

Importance

Users with limited technical knowledge should use it easily.

Relationship with Other Attributes

Accessibility: Improves ease of use

Performance: Faster systems feel easier

Customer Satisfaction: Easy interface builds trust

Example

Simple navigation from Home → Register → Login → Eligibility

5. Accessibility

Definition

Accessibility ensures the system can be used by people of different languages and skills.

Importance

The platform must support rural and non-English users.

Relationship with Other Attributes

Usability: Accessible systems are easier

Customer Satisfaction: Inclusive systems benefit more users

Performance: Lightweight design helps slow internet users

Example

Multilingual support such as English, Hindi, Marathi

6. Maintainability

Definition

Maintainability is the ease of updating the system and fixing bugs.

Importance

Government schemes change frequently.

Relationship with Other Attributes

Reliability: Fixing bugs improves correctness

Security: Updates improve protection

Scalability: Easier expansion

Example

Modular Flask structure with separate components

7. Scalability

Definition

Scalability is the ability to handle increasing users and data.

Importance

More users will access the system over time.

Relationship with Other Attributes

Performance: Maintains speed under load

Availability: Prevents crashes

Maintainability: Supports expansion

Example

Handling high traffic during scholarship periods

8. Availability

Definition

Availability ensures the system is accessible at all times.

Importance

Users may need information anytime.

Relationship with Other Attributes

Reliability: Always working systems are trusted

Performance: Should not slow down

Security: Protection ensures uptime

Example

24/7 system uptime

9. Interoperability

Definition

Interoperability is the ability to work with external systems.

Importance

Future integrations with government services are required.

Relationship with Other Attributes

Security: External systems must be secure

Scalability: Must handle growth

Maintainability: Standard APIs simplify integration

Example

Email service for OTP delivery

10. Customer Satisfaction

Definition

Customer satisfaction reflects user trust and experience.

Importance

Satisfied users will reuse and recommend the system.

Relationship with Other Attributes

Usability: Easy systems increase satisfaction

Performance: Faster response improves experience

Reliability: Correct results build trust

Example

Users successfully finding schemes

Summary Table

Quality Attribute	Definition	Key Relationships
Performance	Speed of system operations	Usability, Scalability, Satisfaction
Security	Protection of data	Reliability, Maintainability
Reliability	Consistent correct output	Availability, Performance
Usability	Ease of use	Accessibility, Satisfaction
Accessibility	Inclusive access	Usability, Performance
Maintainability	Ease of updates	Reliability, Security
Scalability	Handles growth	Performance, Availability
Availability	System uptime	Reliability, Security
Interoperability	Works with external systems	Security, Scalability
Customer Satisfaction	User experience	Usability, Reliability

Conclusion

The quality attributes identified for Sarkari Sahayata are essential for building a reliable and user-friendly system. Since the platform connects citizens with government schemes, it must provide fast performance, accurate results, strong security, and simple navigation.

These attributes are interconnected. Accessibility improves usability, security improves reliability, and maintainability supports future updates. Scalability ensures the system can handle increasing users without performance loss.

By applying these quality attributes, Sarkari Sahayata becomes a dependable and efficient platform that helps bridge the gap between citizens and government welfare schemes.

EXPERIMENT 11

Aim: Implement quality attributes using McCall's quality factors and criteria.

CO: Learner will be able to Apply the software quality assurance factors to Identify Quality Attributes and their Relationships.

Theory:

What is McCall's Quality Model

McCall's Quality Model is a software quality framework used to evaluate a system using defined quality factors and criteria. It helps measure how effective, secure, reliable, and maintainable a software product is. The model connects user expectations with technical implementation and is widely used in software engineering for quality assurance and system improvement. It helps developers build systems that are efficient, user-friendly, and future-ready.

Focus Areas of McCall's Model

The model focuses on three major areas:

Product Operation: Checks how well the system performs during actual use

Product Revision: Focuses on modification, maintenance, and testing

Product Transition: Checks adaptability, portability, and integration

It ensures a balance between functionality, flexibility, and scalability.

Advantages

- Provides a structured method for evaluating software quality
- Covers important factors such as security, reliability, and usability
- Helps identify system weaknesses early
- Improves long-term maintainability and scalability
- Useful for citizen service platforms

Disadvantages

- Some quality factors may overlap
- Requires detailed testing for implementation
- Measuring all attributes takes time
- Needs proper documentation and planning
- Requires updates as technology evolves

Applications

- Government service portals
- Banking and financial systems
- Healthcare platforms
- Educational portals
- Web applications

Implementation for Sarkari Sahayata

1. Product Operation

a. Correctness

Definition

Ensures the system performs all required functions accurately and produces valid outputs

McCall's Criteria

- Users receive correct scheme recommendations
- Eligibility results match scheme criteria
- Registration and login function correctly
- Multilingual content displays accurate translations

Implementation

- Validate user inputs
- Test eligibility logic with multiple cases

- Verify database scheme data
- Perform end-to-end testing

b. Reliability

Definition

Ability of the system to function consistently without failure

McCall's Criteria

- System should not crash
- OTP verification must work consistently
- Scheme results load correctly
- User data should not be lost

Implementation

- Use exception handling
- Maintain stable database connections
- Test repeated operations
- Implement error logging

c. Efficiency

Definition

Proper utilization of resources while maintaining performance

McCall's Criteria

- Fast page loading
- Quick eligibility results
- Optimized database queries
- Smooth performance on low-end devices

Implementation

- Optimize SQL queries
- Reduce unnecessary API calls
- Use lightweight frontend
- Cache frequently used data

d. Integrity (Security)

Definition

Protection of user data from unauthorized access

McCall's Criteria

- Secure password storage
- Restricted account access
- OTP verification for security
- Protection from malicious input

Implementation

- Use password hashing
- Implement OTP authentication
- Validate and sanitize inputs
- Use secure session handling

e. Usability

Definition

Ease of interaction for users

McCall's Criteria

- Simple navigation
- Easy search and filtering
- Clear registration process

- Suitable for non-technical users

Implementation

- Design clean user interface
- Use readable fonts and forms
- Provide multilingual support
- Improve based on user feedback

2. Product Revision

a. Maintainability

Definition

Ease of updating and fixing the system

McCall's Criteria

- Easy addition of schemes
- Bugs can be traced quickly
- Code is understandable
- Updates do not break features

Implementation

- Use modular Flask structure
- Separate logic and templates
- Maintain documentation
- Use version control

b. Flexibility

Definition

Ability to adapt to future changes

McCall's Criteria

- Add new categories
- Enhance AI features
- Add more languages
- Modify UI independently

Implementation

- Use reusable components
- Separate business logic
- Design scalable database
- Use configurable modules

c. Testability

Definition

Ease of testing the system

McCall's Criteria

- Modules can be tested independently
- Bugs are easy to detect
- Supports automated testing
- Covers user scenarios

Implementation

- Test modules separately
- Use sample datasets
- Perform integration testing
- Maintain bug tracking

3. Product Transition

a. Portability

Definition

Ability to run on different environments

McCall's Criteria

- Works on mobile and desktop
- Supports different browsers
- Easy deployment
- Responsive design

Implementation

- Use responsive design
- Test on multiple browsers
- Deploy using Flask servers
- Optimize UI for devices

b. Interoperability

Definition

Ability to interact with external systems

McCall's Criteria

- Email OTP integration
- Future API compatibility
- Smooth data exchange
- Secure external connections

Implementation

- Use SMTP for emails
- Use standard APIs
- Maintain structured database
- Ensure secure communication

Conclusion

McCall's Quality Model helps in evaluating and improving Sarkari Sahayata systematically. Product Operation ensures correct, secure, and user-friendly behavior. Product Revision ensures the system remains maintainable, flexible, and testable. Product Transition ensures compatibility and future integration.

Applying these quality factors makes Sarkari Sahayata a reliable, scalable, and citizen-focused digital platform.